Day 7 Evidence Workbooklet — Instructor Key

Ordered Validation and First-True Behavior

Engineering practice → ordered rules → first true branch → skipped branches → support plan

Instructor key. Same layout as student copy; solutions filled in response boxes. Print format: duplex, left-stapled packet.

Name		**Section**		**Date**	
Score	_____/42		Mastery check	secure / developing / needs support	

The sensor-kit checkout table now uses several rules in a fixed order. A kit might have more than one issue, but the branch outcome should identify the first action the team should take. Today the focus is ordered validation: trace which condition is first true and explain why later branches are skipped.

Engineering task	Evaluate a sensor kit through an ordered checkout policy without losing evidence about skipped issues.
Computational move	Read an `if/elif/else` chain as an ordered list of checks where the first true branch wins.
Python focus	`if`, `elif`, `else`, first-true behavior, skipped branches, comparison and string-label checks.
Evidence to keep	rule order, condition result, first true branch, skipped later branch, branch outcome assigned, and rule-order risk.

Day 7 working rules

1. Python checks an `if/elif/else` chain from top to bottom.
2. The first true branch runs.
3. Later branches are skipped after the first true branch.
4. Rule order can change the branch outcome.
5. A reliable branch-decision note names the first true rule and any important skipped issue.

1 Read the ordered rules before tracing cases

recognize predict

The checkout desk now uses more than one rule. Python checks the rules in order. The first true rule assigns the branch outcome and the later rules are skipped.

```
kit_label = "S-22"
charge_percent = 76
calibration_label = "current"
borrower_initials = "MR"
damage_note = ""

if damage_note != "":
    action = "repair bench"
elif charge_percent < 80:
    action = "charge station"
elif calibration_label != "current":
    action = "calibration bench"
elif borrower_initials == "":
    action = "borrower log needed"
else:
    action = "release kit"
```

Pts.	Prompt	My evidence
1	Which rule is checked first?	**Key:** `damage_note != ""`.
1	Which rule is checked second?	**Key:** `charge_percent < 80`.
1	Which rule is true for this kit?	**Key:** `charge_percent < 80` is true because `76 < 80`; the damage rule is false.
1	Which branch outcome is assigned?	**Key:** `charge station`.

2 Trace first-true behavior

trace explain

Complete the table. Stop at the first true rule for each case.

Pts.	Damage?	Charge	Calibration	Initials	First true rule and branch outcome
2	no	76	current	MR	**Key:** First true rule: `charge_percent < 80`; branch outcome: `charge station`.
2	cracked case	76	expired	MR	**Key:** First true rule: `damage_note != ""`; branch outcome: `repair bench`. Later charge and calibration issues are skipped for selection.
2	no	91	expired	MR	**Key:** First true rule: `calibration_label != "current"`; branch outcome: `calibration bench`.
2	no	91	current	blank	**Key:** First true rule: `borrower_initials == ""`; branch outcome: `borrower log needed`.

First-true reminder

If an early rule is true, Python does not continue looking for a later, also-true rule. Your evidence should name the first true rule, not just every possible problem.

3 Explain skipped branches

explain debug

A kit may have more than one problem. Ordered selection decides which problem is handled first.

```
charge_percent = 72
calibration_label = "expired"
damage_note = ""

if damage_note != "":
    action = "repair bench"
elif charge_percent < 80:
    action = "charge station"
elif calibration_label != "current":
    action = "calibration bench"
else:
    action = "release kit"
```

Pts.	Prompt	My response
2	Which branch outcome is assigned?	**Key:** `charge station`.
2	Is the calibration rule also true for this kit? Explain.	**Key:** Yes. `calibration_label != "current"` is true because the label is `expired`.
2	Why does the calibration branch not run?	**Key:** The earlier low-charge rule is true first, so Python runs `charge station` and skips later branches.
2	What would be dangerous about reporting only the final branch outcome without the skipped issue?	**Key:** The team might charge the kit and then miss that calibration is still expired, so the kit could be treated as ready too soon.

4 Find a rule-order problem

debug test

A teammate proposes moving the release check above the damage check.

```
if charge_percent >= 80 and calibration_label == "current" and borrower_initials != "":
    action = "release kit"
elif damage_note != "":
    action = "repair bench"
else:
    action = "hold for review"
```

Pts.	Prompt	My response
2	What important condition is checked too late?	**Key:** The damage check, `damage_note != ""`.
2	Give a kit case that would be assigned unsafely by the proposed order.	**Key:** A damaged but otherwise ready kit: `damage_note = "cracked case"`, `charge_percent = 90`, `calibration_label = "current"`, and initials present.
2	What branch outcome should that case receive?	**Key:** `repair bench`.
2	Write one sentence explaining the rule-order risk.	**Key:** If the release rule is checked before damage, a damaged but otherwise ready kit can be released before the repair rule is reached.

5 Constrained edit of one rule

implement debug

Only edit the calibration rule below. Do not rewrite the whole decision chain.

```
    elif ________________________________________________:
        action = "calibration bench"
```

Pts.	Prompt	My response
3	Complete the calibration rule.	**Key:** `calibration_label != "current"`.
2	Explain why the rule should compare to `"current"`.	**Key:** The policy needs the exact acceptable label. Anything other than `current`, such as `expired`, should produce calibration.
2	Give one calibration label that should produce the calibration bench.	**Key:** `expired`.
1	Name one later branch that would be skipped if this rule is true.	**Key:** The borrower-log branch, `borrower_initials == ""`, would be skipped.

6 Transfer and support plan

transfer explain

Pts.	Ordered-validation note
4	Write a note that explains how ordered validation differs from a single `if/else`. Use first true and skipped branch in your explanation. **Key:** A single `if/else` chooses between two branch outcomes from one condition. Ordered validation checks several rules top to bottom; the first true rule wins, so a low-charge and expired kit produces `charge station` while the calibration branch is skipped for that pass.
2	Circle the support plan you would use next: rule order / skipped branch / comparison / string label / written explanation. Name one next action: _____

Bridge to Day 8

Day 8 uses expected-vs-actual evidence to debug a branch decision when the selected action does not match the rule intention.

7 Lab bridge: complete the body of a provided wrapper

VIP 18.8 | Instructor Key

What stays fixed

The correct body uses the boundaries 8 and 16. Keep the wrapper, parameter names, and exact strings fixed. Students complete only the decision body; the page is unscored.

```
def badge_sheet_plan(group_count, badges_per_group):
    total_badges = group_count * badges_per_group
    if total_badges <= 8:
        return "one sheet"
    elif total_badges <= 16:
        return "two sheets"
    else:
        return "print batch"
```

Total badges	Expected branch outcome	First true condition	Boundary evidence
8	one sheet	**Key:** `total_badges <= 8`	**Key:** 8 is included in the first branch outcome.
9	two sheets	**Key:** First rule false; `total_badges <= 16` true.	**Key:** 9 is the first value above one-sheet capacity.
16	two sheets	**Key:** First rule false; `total_badges <= 16` true.	**Key:** 16 is included in the second branch outcome.
17	print batch	**Key:** Both comparisons false; `else` runs.	**Key:** 17 is the first value above two-sheet capacity.

Exact-output contract

Accept only the task's exact labels: `"one sheet"`, `"two sheets"`, and `"print batch"`. The visible checks treat wording as part of the output contract.

Bridge Day 7 VIP Evidence Handoff

INSTRUCTOR KEY • Contract 07a04826c975 • Accessible v5 successor

Why engineers use this page

Carry comparison and branch-order evidence into the current top-level decision cells; Activity 18.14 supplies its loop.

Decision boundary: Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

Construct readiness before independent work

New authoring tools: comparison expression, boolean operator, if elif else, abs call, counter update
Carried authoring tools: assignment, print call, numeric literal, arithmetic expression, input call, int conversion, float conversion, f string
Supplied-code exposure only: for loop, list traversal
An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

Notebook cells and task constructs

Activity	Work	Stable cells	Required authoring constructs
18.13	Compare With Tolerance	`18-13-work / 18-13-transfer / 18-13-cold`	comparison expression, f string, float conversion, input call
18.14	Count High Readings	`18-14-work / 18-14-transfer / 18-14-cold`	arithmetic expression, comparison expression, counter update, f string, if elif else
18.15	Lamination Fee	`18-15-work / 18-15-transfer / 18-15-cold`	comparison expression, f string, if elif else, input call, int conversion
18.16	Temperature Status	`18-16-work / 18-16-transfer / 18-16-cold`	comparison expression, f string, float conversion, if elif else, input call
18.17	Pickup Window	`18-17-work / 18-17-transfer / 18-17-cold`	boolean operator, comparison expression, f string, if elif else, input call, int conversion

Readiness evidence

1. Identify which activity supplies a loop and state exactly what decision you are responsible for writing inside it.

   Model response: Activity 18.14 supplies the loop. Students write the strict greater-than decision and update the counter only on matching passes.

2. For one of Activities 18.13, 18.15, 18.16, or 18.17, name an equality boundary that must receive its own test.

   Model response: Accept a correct activity boundary, including the tolerance equality in 18.13, an inclusive fee boundary in 18.15, the exact +5 boundary in 18.16, or the quick-pickup time boundary in 18.17.

Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** _____ **My help trigger:** _____

Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

Evidence field	Record
Prediction	A specific expected state or output before execution.
Observed Result	The actual state or output, including a value or exact text.
Revision Reason	The defect or mismatch and the change that addresses it.
Engineering Meaning	How the evidence changes or supports the engineering decision.
Units Or Not Applicable	The physical unit, or the evidence type when no unit applies.
Assumption	One condition accepted as true for this model or rule.
Model Limit	One situation the result does not justify or predict.
Next Test	A concrete boundary, invalid, or alternate case with an expected result.
Help Trigger	The exact unresolved point that would trigger a targeted question.
Minutes Used	Approximate minutes from first prediction through revision.

Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.